

网页文章错误检测 Agent —— 技术介绍文档

本文档面向团队所有成员，系统介绍 Agent 推理核心的设计思路、内部架构与接口规范。

目录

- [背景与目标](#)
- [Agent 在系统中的定位](#)
- [整体处理流程](#)
- [核心推理框架](#)
 - [4.1 Plan-and-Execute 外层框架](#)
 - [4.2 ReAct 内层循环](#)
- [模块详解](#)
 - [5.1 文章预处理 \(preprocess.py\)](#)
 - [5.2 声明扫描器 \(scanner.py\)](#)
 - [5.3 规划器 \(planner.py\)](#)
 - [5.4 Agent 主类 \(agent.py\)](#)
 - [5.5 LLM 客户端层 \(llm/\)](#)
 - [5.6 工具链 \(tools/\)](#)
- [SSE 事件流设计](#)
- [数据结构](#)
- [技术选型说明](#)
- [运行与部署](#)

1. 背景与目标

互联网文章中存在大量未经核实的声明——包括错误数据、伪造引用、逻辑谬误和内部矛盾。人工逐一核查效率极低。

本项目构建了一个浏览器插件，让用户在阅读网页时可以一键触发自动事实核查。Agent 是这个插件的推理大脑，负责：

- 从文章中识别出哪些句子值得核查
- 对每条可疑声明，制定验证策略并调用外部工具取证
- 给出结论（错误类型 + 置信度 + 推理依据 + 证据链接）
- 将中间推理过程实时流式传输给前端展示

Agent 不做的事：渲染高亮、管理用户账号、存储历史记录——这些都由其他模块负责。

2. Agent 在系统中的定位



Agent 以 **Python 包**形式被 FastAPI 直接 import，不拆微服务。后端消费 Agent 的 `async generator`，将每一个事件包装成 SSE 帧向前端推送。

3. 整体处理流程

以一篇文章为输入，Agent 的完整处理流程如下：





4. 核心推理框架

4.1 Plan-and-Execute 外层框架

Plan-and-Execute 是一种**先整体规划，再逐步执行**的 Agent 架构。

与"每一步都重新决定下一步"的纯 ReAct 相比，Plan-and-Execute 的优势在于：

- **效率**：一次性确定所有声明，避免重复扫描文章
- **可控**：前端在验证开始前就能得到 `PlanEvent`，展示进度条
- **可排序**：按可疑度优先处理重要声明

在本项目中，Plan-and-Execute 的"计划"阶段由 `scanner` + `planner` 完成，"执行"阶段由 `_react_loop` 逐条完成。

```
Plan 阶段（一次性）：
    scan_article → build_plan → emit PlanEvent
```

```
Execute 阶段（循环）：
    for claim in plan.claims:
        async for event in _react_loop(claim):
            yield event
```

4.2 ReAct 内层循环

ReAct (**R**easoning + **A**cting) 是一种让 LLM 交替推理和行动的范式，每一步格式如下：

```
Thought: <当前推理>
Action: <工具名>
Action Input: <工具输入>
--- 执行工具 ---
Observation: <工具返回>
Thought: <基于观察的下一步推理>
...
Final Answer: {"error_type": ..., "confidence": ..., "reasoning": ...,
"evidence_urls": [...]}
```

关键设计决策：

问题	决策
LLM 怎么知道有哪些工具？	System Prompt 列出所有工具名和 description
什么时候停止循环？	LLM 输出 <code>Final Answer</code> 自行决定，或达到 <code>MAX_STEPS=6</code> 兜底
messages 怎么管理？	追加式 multi-turn list, Observation 作为 user 角色追加
工具失败怎么办？	异常信息作为 Observation 传回 LLM，循环不中断，同时 yield <code>ErrorEvent</code>
输出格式违规怎么办？	yield <code>ErrorEvent</code> ，向 LLM 发送格式提示，继续循环
超过步数未结论？	yield <code>confidence=0.0</code> 、 <code>reasoning="达到最大推理步数"</code> 的兜底 <code>AnnotationEvent</code>

为什么要求“至少调用一次工具”？

在 user 消息末尾加了约束：“你必须至少调用一次工具进行外部核实，不得直接给出 `Final Answer`”。这是为了防止 LLM 仅凭训练数据就做出结论，确保每条声明都经过外部检索。

Final Answer 的 JSON 结构：

```
{
  "error_type": "factual_error" | "logical_fallacy" | "contradiction" |
  "unsupported_claim" | null,
  "confidence": 0.0 ~ 1.0,
  "reasoning": "中文推理摘要（展示在 Tooltip）",
  "evidence_urls": ["https://..."]
}
```

`error_type=null` 表示未发现错误，`confidence` 反映把握程度，合法值在代码中强制校验。

5. 模块详解

5.1 文章预处理（preprocess.py）

职责：将原始 HTML 正文处理成带精确字符偏移量的结构化句子列表。

为什么需要偏移量？

前端高亮依赖的是（`start_offset`，`end_offset`）这对字符位置——浏览器需要知道在原始文本的第几个字符到第几个字符之间画高亮。偏移量必须在预处理阶段就保留，不能在扫描后再重新计算。

处理步骤：

原始文本

- ↳ `extract_paragraphs()` 按空行分段，记录每段 `start/end`
 - ↳ `split_sentences()` 按。！？分句，处理.....省略号特殊情况
 - ↳ 噪声过滤
 - └ 过滤 < 5 字符的片段
 - └ 过滤纯数字行
 - └ 过滤无标点且 ≤ 20 字的疑似标题行

输出结构：

```
ProcessedArticle(
  raw_text: str,
  paragraphs: list[ProcessedParagraph],
  sentences: list[ProcessedSentence] # 每句都有 start_offset, end_offset
)
```

5.2 声明扫描器（scanner.py）

职责：从文章中识别"值得事实核查"的声明，返回带偏移量的 `Claim` 列表。

为什么用 LLM 而不是规则？

"值得核查"是一个语义判断——涉及具体事实、引用、因果断言。正则表达式无法捕捉语义，LLM 更适合。

处理策略：

- 文章 < 8000 字：整篇一次发给 LLM
- 文章 ≥ 8000 字：按段落批次分批处理，避免超出 context window

LLM 提示词约束（哪些句子值得提取）：

- 1. 包含具体数字或统计数据
- 2. 描述具体历史事件或时间节点
- 3. 引用他人观点或研究结论
- 4. 做出因果关系断言

防幻觉机制：LLM 只负责返回声明文本，不负责偏移量。偏移量通过 `str.find()` 在原文中精确定位。如果 `find()` 返回 -1 (LLM 捏造了原文中不存在的句子)，该声明直接丢弃。

```
# 核心逻辑
claim_texts = await llm_extract(article_text)      # LLM 返回文本列表
claims = []
for text in claim_texts:
    offset = article.find(text)
    if offset == -1:
        continue # 丢弃虚构
    claims.append(Claim(id=f"c{idx:03d}", text=text,
                       position=(offset, offset + len(text)), ...))
```

5.3 规划器 (planner.py)

职责：对扫描器输出的声明计算"可疑度分数"，按分数降序排列，决定验证优先级。

为什么不用 LLM 排序？

规划器是高频操作（每篇文章都要跑一遍），且排序标准是固定的启发式规则，没有必要消耗 LLM 调用。规则评分既快又可解释。

评分规则（各项独立累加，最终裁剪到 `[0.0, 1.0]`）：

条件	分值
含具体数字或百分比 (<code>\d+</code> / <code>%</code> / <code>%</code>)	+0.35
含引用标志词 (据、根据、研究显示、报告指出 等)	+0.20
含绝对断言词 (唯一、最、全球、史上、首个 等)	+0.20
含时间节点 (年/月/日)	+0.15
声明文本过短 (< 8 字)	-0.15

分数越高 = 越优先验证。

5.4 Agent 主类 (agent.py)

职责：串联所有组件，实现 Plan-and-Execute + ReAct 完整推理流程，作为 `AsyncGenerator` 对外暴露。

对外接口：

```
agent = Agent(complex_llm=ClaudeClient())

async for event in agent.run(article_text):
    # event 是 AgentState (7 种事件之一)
    yield f"data: {event.model_dump_json()}\n\n"
```

内部组成：

方法	职责
<code>run(article_text)</code>	主入口，Plan-and-Execute 外层
<code>_react_loop(claim)</code>	ReAct 内层循环，处理单条声明
<code>_build_system_prompt()</code>	构建包含工具列表和格式规范的系统提示词
<code>_parse_react_response(text)</code>	正则解析 LLM 输出，识别 action/final/unknown

错误隔离原则：

- 单个工具失败 → `ErrorEvent` + 继续循环
- 单条声明的 `_react_loop` 异常 → `ErrorEvent` + 继续下一条声明
- `scan_article` 失败 → `ErrorEvent` + `DoneEvent(0)` + 提前返回
- `prepare_cross_reference_context` 失败 → `StatusEvent` 提示跳过，**不影响主流程**

5.5 LLM 客户端层 (llm/)

采用**抽象基类 + 双实现**的设计，方便后续替换模型。

```
BaseLLMClient (abstract)
├─ complete(messages) -> str          # 普通对话
└─ complete_with_tools(messages, tools) -> dict # 工具调用

    └─ ClaudeClient      ← Sonnet 4.6, 复杂推理 (扫描 + ReAct)
    └─ GLMClient         ← GLM-4-Flash, 简单任务 (不支持 tool use)
```

为什么用两个模型？

场景	模型	理由
文章扫描、ReAct 推理	Claude Sonnet 4.6	需要强推理能力和长上下文
简单辅助任务	GLM-4-Flash	速度快、成本低、OpenAI 兼容接口

`ClaudeClient` 支持配置 `ANTHROPIC_BASE_URL`（当前指向 `yunwu.ai` 代理），方便在不同网络环境下运行。

5.6 工具链 (tools/)

工具是 ReAct 循环中 Agent 与外部世界交互的唯一途径。

工具规范 (所有工具必须遵守) :

```
async def tool_func(input: str) -> str:
    # 所有异常必须在内部捕获, 不向外抛出
    # 返回字符串结果
```

工具注册表:

```
TOOL_REGISTRY: dict[str, ToolSpec] = {
    "web_search": ToolSpec(name, description, input_schema, func),
    ...
}
```

当前已注册的 4 个工具:

web_search

- **用途:** 在互联网搜索声明相关信息
- **输入:** 搜索关键词字符串
- **输出:** 结论: {answer}\n来源: {urls}
- **降级策略:** 优先 Tavily API, 无 Key 则降级到 DuckDuckGo (无需 Key)

wikipedia_lookup

- **用途:** 查询维基百科条目摘要
- **输入:** 词条名称
- **输出:** 摘要文本
- **降级策略:** 先查中文维基, 无结果再查英文维基

source_verifier

- **用途:** 抓取指定 URL 的网页正文, 用于核实声明来源
- **输入:** URL
- **输出:** 正文文本 (截断至 2000 字)
- **实现:** httpx 异步请求 + readability 提取正文

cross_reference

- **用途:** 检测声明与文章内其他句子的语义矛盾 (内部一致性检测)
- **输入:** 待验证声明文本
- **输出:** 相关句列表 + 矛盾判断结果
- **技术:** paraphrase-multilingual-MiniLM-L12-v2 向量模型 (~420MB), 句向量点积相似度, 矛盾由 LLM 最终判断
- **注意:** 调用前需先调用 prepare_cross_reference_context(article_text, llm) 初始化文章上下文

6. SSE 事件流设计

Agent 通过 `AsyncGenerator` 实时 `yield` 事件，后端包装为 Server-Sent Events 推送给前端。

事件类型总览：

事件类型	触发时机	主要接收方
<code>status</code>	各阶段开始/结束	前端进度条、调试日志
<code>plan</code>	扫描规划完成	前端渲染待验证声明列表
<code>thinking</code>	每个 ReAct Thought 步骤	前端"推理过程"展示
<code>tool_call</code>	每次工具调用完成	前端"调用工具"展示
<code>annotation</code>	单条声明验证完成	前端高亮 、评估平台记录
<code>error</code>	工具失败、LLM 异常	前端警告、日志
<code>done</code>	所有声明处理完毕	前端关闭进度条

典型事件序列（处理含 3 条声明的文章）：

```
status(scan 开始)
status(扫描完成, 3 条声明)
status(准备交叉引用上下文)
status(交叉引用上下文准备完成)
plan(total=3, claims=[...])

status(开始验证 c001)
thinking(c001, "需要搜索该数据的来源")
tool_call(c001, web_search, "搜索词", "搜索结果...")
thinking(c001, "搜索结果与声明不符")
annotation(c001, error_type="factual_error", confidence=0.85, ...)

status(开始验证 c002)
thinking(c002, ...)
tool_call(c002, wikipedia_lookup, ...)
annotation(c002, error_type=null, confidence=0.9, ...)

...

status(分析完成, 1 条错误标注)
done(total_annotations=1)
```

每帧 SSE 格式：

```
data: {"type": "annotation", "claim_id": "c001", "text": "...", ...}\n\n
```

7. 数据结构

内部结构（不发送给前端）

```
@dataclass
class Claim:
    id: str                # "c001", "c002", ...
    text: str              # 原文片段
    position: tuple[int, int] # (start_offset, end_offset)
    suspicion_score: float  # 0.0 ~ 1.0

@dataclass
class VerificationPlan:
    claims: list[Claim]      # 按可疑度降序
    status: dict[str, str]  # claim_id → "pending|running|done|error"
```

SSE 事件（Pydantic 模型，发送给前端）

```
# AnnotationEvent 是最关键的事件
class AnnotationEvent(BaseModel):
    type: Literal["annotation"]
    claim_id: str
    text: str
    start_offset: int
    end_offset: int
    error_type: "factual_error" | "logical_fallacy" | "contradiction" |
"unsupported_claim" | None
    confidence: float      # 0.0 ~ 1.0
    reasoning: str         # 中文推理摘要，前端 Tooltip 展示
    evidence_urls: list[str]
```

error_type 含义：

值	含义
factual_error	事实性错误（数字、日期、人名等与事实不符）
logical_fallacy	逻辑谬误（因果倒置、以偏概全等）
contradiction	与文章内其他内容矛盾
unsupported_claim	无法找到支撑或反驳证据（存疑）
null	未发现错误

8. 技术选型说明

为什么选 ReAct 而不是纯 Function Calling?

Claude 的 Function Calling (tool_use API) 在每次调用后强制返回工具调用请求, 无法像 ReAct 那样在每一步都输出推理过程 (Thought)。ReAct 的文本格式更灵活, 且 Thought 内容可以直接 yield 给前端展示, 让用户看到 Agent 的推理过程。

为什么 Prompt 而不是结构化 Tool Use API?

ReAct 的核心是"文本格式的 Thought-Action-Observation 循环", 而不是 API 层面的结构化调用。使用纯文本格式:

- 可以在同一次 LLM 调用中同时输出 Thought 和 Action
- 工具选择逻辑完全在提示词中描述, 调试更直观
- 对 GLM 等不支持 tool use API 的模型也兼容

为什么工具输出截断 500 字符再 yield?

`ToolCallEvent` 的 `tool_output` 字段只截断给前端展示, 完整内容会追加进 `messages` (`Observation: {完整输出}`) 供 LLM 决策。截断是为了避免 SSE 帧过大导致前端渲染卡顿。

为什么 cross_reference 单独初始化而不在工具函数内部?

`paraphrase-multilingual-MiniLM-L12-v2` 模型加载需要数秒时间, 且每篇文章的句子向量只需计算一次。如果每次工具调用都重新加载和计算, 延迟将难以接受。

`prepare_cross_reference_context()` 在验证开始前预热, 将结果缓存在 `CrossReferenceService` 单例中。

9. 运行与部署

环境准备

```
# 安装依赖
pip install -r requirements.txt

# 配置 API Keys
cp .env.example .env
# 编辑 .env, 填入:
#   ANTHROPIC_API_KEY=...
#   ANTHROPIC_BASE_URL=https://yunwu.ai (可选, 代理地址)
#   ZHIPUAI_API_KEY=... (GLM-4-Flash)
#   TAVILY_API_KEY=... (可选, web_search 优先使用)
```

启动后端

```
cd agent-core
uvicorn backend.main:app --reload --port 8000
```

运行测试

```
pytest tests/ -v                                # 全部测试（当前 32 个，全部通过）
pytest tests/test_agent.py -v                   # 仅 Agent 测试
pytest tests/test_scanner.py -v                 # 仅扫描器测试
```

预热 cross_reference 向量模型

首次运行时需要下载 ~420MB 的模型文件：

```
python -m agent.tools.cross_reference
# 模型缓存到 .cache/huggingface/
# 可通过 CROSS_REFERENCE_CACHE_DIR 环境变量修改缓存目录
# 离线环境：设置 LOCAL_MODEL_PATH 指向本地模型路径
```

手动测试 Agent

```
python - <<'EOF'
import asyncio
from agent import Agent
from agent.llm import ClaudeClient

async def main():
    agent = Agent(complex_llm=ClaudeClient())
    async for event in agent.run("苹果公司成立于1977年，总部位于美国德克萨斯州。"):
        print(event.model_dump_json())

asyncio.run(main())
EOF
```

附录：项目文件结构

```
agent-core/
├─ agent/
│   ├── __init__.py      # 导出 Agent, AgentState
│   ├── models.py        # 数据结构与 SSE 事件定义
│   ├── agent.py         # Agent 主类（Plan-and-Execute + ReAct）
│   ├── scanner.py       # 声明扫描器
│   ├── planner.py       # 可疑度规划器
│   ├── preprocess.py    # 文章预处理管道
│   └─ llm/
│       ├── base.py      # BaseLLMClient 抽象基类
│       ├── claude.py    # Claude Sonnet 4.6 客户端
│       └─ glm.py        # GLM-4-Flash 客户端
│   └─ tools/
│       ├── registry.py  # 工具注册表
│       ├── web_search.py # 网络搜索（Tavily / DuckDuckGo）
│       ├── wikipedia_lookup.py # 维基百科查询
│       ├── source_verifier.py # 网页正文抓取
│       └─ cross_reference.py # 文章内部矛盾检测（向量检索）
└─ backend/
```

```
|   └─ main.py                # FastAPI SSE 接口
├─ tests/                    # 32 个单元测试（全部通过）
└─ docs/
    ├─ agent-introduction.md  # 本文档
    ├─ agent-core-interface.md # Agent 核心接口文档
    └─ cross-reference-tool.md # cross_reference 工具详细说明
```